

EDMS 657 HW4 R

Alex Caramanico

2025-11-20

Multivariate Mixture Modeling

We will once again use the Iris dataset (Iris-Data.dat). Use the `mclust()` functions in R to fit a mixture with 1:4 latent classes. Using BIC, provide the best fitting solution in terms of (1) the number of classes based on BIC and (2) any constraints on the covariance structure needed (see Fraley (2010) for a description and through our slides in class). Explain what your results imply about the means and variances and covariances among the classes. A graph may or may not be helpful to support your findings and explanation.

```
library(tidyverse)
library(pvclust)
library(fpc)
library(psych)
library(factoextra)
library(mclust)
iris <- read_table("C:/Users/jacma/OneDrive/school work/UMD/EDMS 657/Homework - Mplus/Iris-Data.dat",
                  col_names = c("sepal_length", "sepal_width",
                                "petal_length", "petal_width",
                                "species"))
iris$species <- as.factor(iris$species)
results <- Mclust(as.matrix(iris[,1:4]), G=1:4)
results$BIC
```

```
## Bayesian Information Criterion (BIC):
```

```
##      EII      VII      EEI      VEI      EVI      VVI      EEE
## 1 -1804.085 -1804.0854 -1522.1202 -1522.1202 -1522.1202 -1522.1202 -829.9782
## 2 -1123.412 -1012.2352 -1042.9679 -956.2823 -1007.3082 -857.5515 -688.0972
## 3 -878.765 -853.8144 -813.0504 -779.1566 -797.8342 -744.6382 -632.9647
## 4 -893.614 -812.6048 -827.4036 -748.4529 -837.5452 -751.0198 -646.0258
##      VEE      EVE      VVE      EEV      VEV      EVV      VVV
## 1 -829.9782 -829.9782 -829.9782 -829.9782 -829.9782 -829.9782 -829.9782
## 2 -656.3270 -657.2263 -605.1841 -644.5997 -561.7285 -658.3306 -574.0178
## 3 -605.3982 -666.5491 -636.4259 -644.7810 -562.5522 -656.0359 -580.8396
## 4 -604.8371 -705.5435 -639.7078 -699.8684 -602.0104 -725.2925 -630.6000
##
## Top 3 models based on the BIC criterion:
##      VEV,2      VEV,3      VVV,2
## -561.7285 -562.5522 -574.0178
```

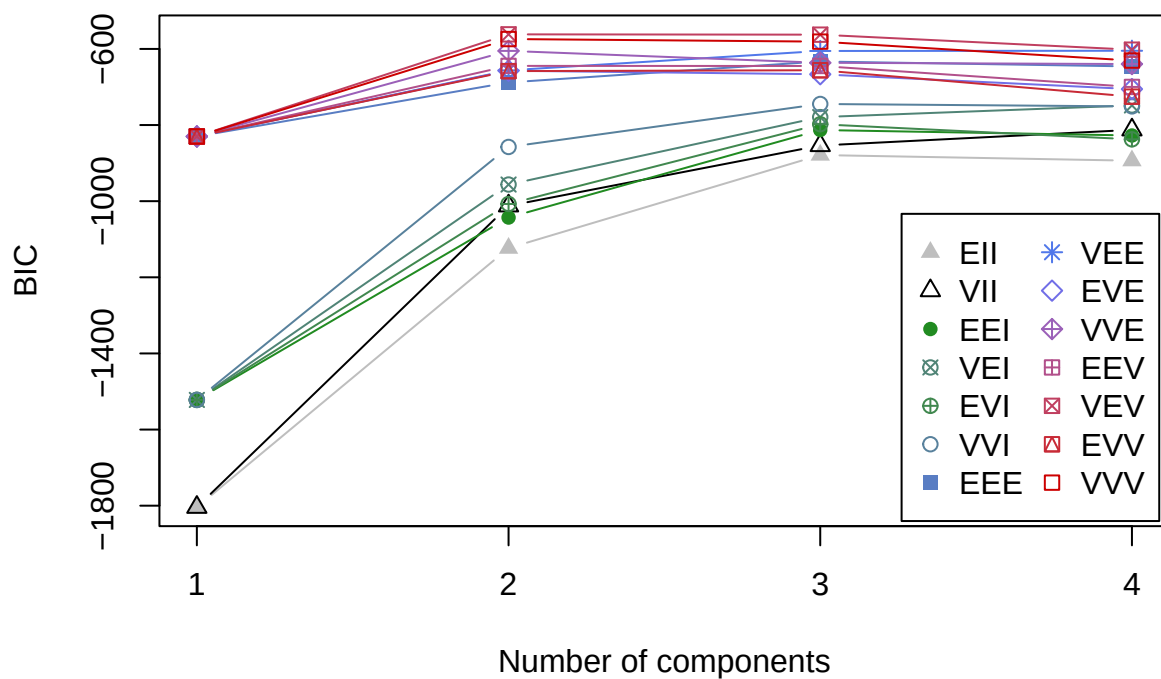
```
results$parameters$variance$sigma
```

```
## , , 1
##
##          sepal_length sepal_width petal_length petal_width
## sepal_length  0.15065114 0.13080115  0.02084463  0.01309107
## sepal_width   0.13080115 0.17604529  0.01603245  0.01221458
## petal_length  0.02084463 0.01603245  0.02808260  0.00601568
## petal_width   0.01309107 0.01221458  0.00601568  0.01042365
##
## , , 2
##
##          sepal_length sepal_width petal_length petal_width
## sepal_length  0.4000438 0.10865444  0.3994018  0.14368256
## sepal_width   0.1086544 0.10928077  0.1238904  0.07284384
## petal_length  0.3994018 0.12389040  0.6109024  0.25738990
## petal_width   0.1436826 0.07284384  0.2573899  0.16808182
```

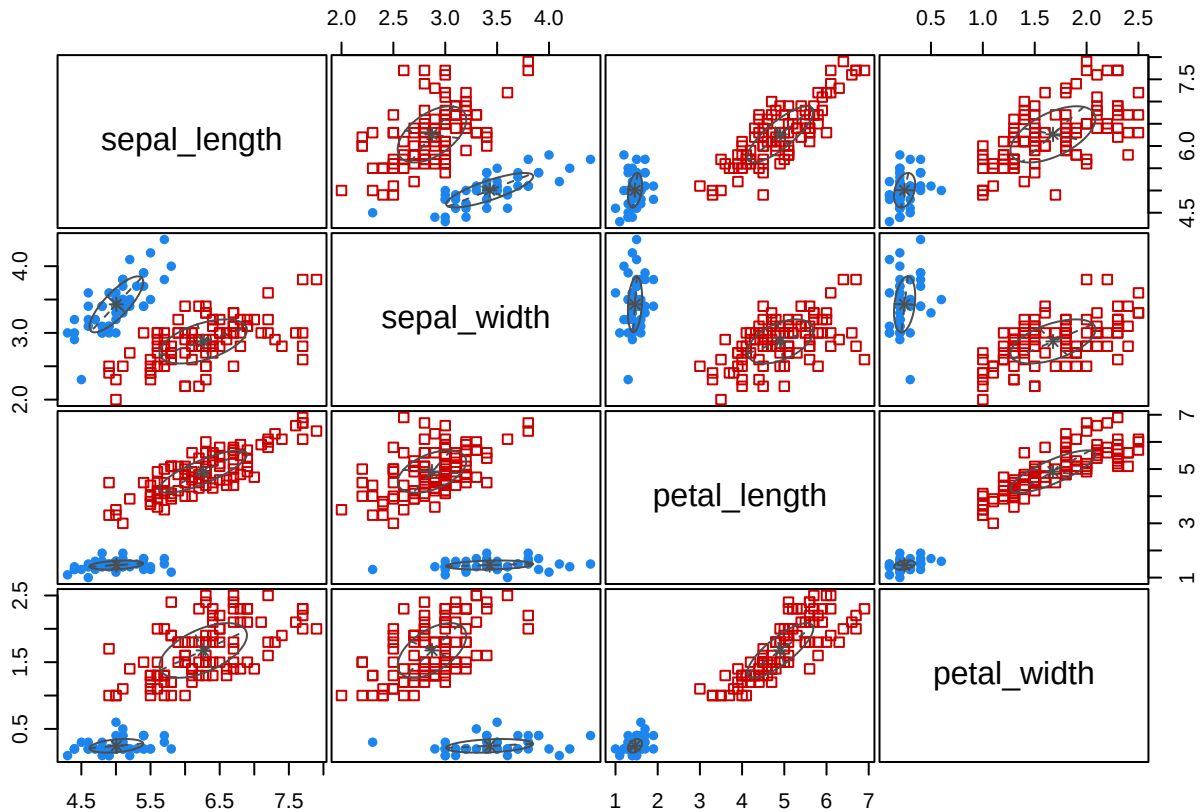
```
results$parameters$mean
```

```
##          [,1]      [,2]
## sepal_length 5.0060022 6.261996
## sepal_width  3.4280049 2.871999
## petal_length 1.4620007 4.905992
## petal_width  0.2459998 1.675997
```

```
plot(results, what="BIC")
```



```
plot(results, what="classification")
```



```
rm(iris, results)
```

Depending upon the covariance structure, the best fitting models use 2 or 3 clusters. Among the 14 structures, the 3-class solution “wins” 6, the 4-class wins 3, and the 2-class wins 5. The best overall fit is the 2-cluster VEV specification, which corresponds to varying volume, equal shape, and varying orientation. The one-cluster solution is by far the worst option for this data, while EII, VII, EII, EVI, and VVI performed particularly poorly in terms of covariance structure.

From the output following the BIC results, we see that the means and variances of each variable and their covariances all vary between the two classes. Non-zero and class-varying covariances make sense because they allow the models to capture additional sources of variability that potentially determine clusters and give each cluster their own unique “shape”. While this is certainly the most flexible specification, it comes with the downside of being more computationally expensive and perhaps more difficult to interpret, especially when there are many classes. This however would not be a concern for this problem because the data is relatively small and a two-class solution is reasonable.

Looking more closely at the covariances matrices, we see that the variances across clusters are very different. E.g. petal_length has a variance of 0.028 in class 1, but 0.6109 in class 3, meaning that the first class has a very “consistent” level of variation in petal_length. Or in other words, the petal length of flowers in this group are all nearly the same, but in group 3 there is a wider range of lengths. Looking at the covariances, we see some differences as well. Notably, sepal_length is much more strongly correlated with petal_length in class 2 than in class 1 (0.399 and 0.0208, respectively), meaning these variables are more closely related in the second class than in the first. That is, in class 2, we’d be more likely to expect a larger sepal_length if provided the flower has a large petal_length, but for class 2, there is no indication on either when the other is given.

The first plot reiterates the findings from the BIC summary table - where the 1-cluster solution is far worse regardless of covariance structure. We also see certain structures performing much better than others, mainly those with variable orientation. This is affirmed in the second plot, which shows the importance of varying orientation in the clustering. The data (and the fitted circles denoting the clusters) are clearly spread in different directions, hence the better performance of an orientation-varying clustering model.

Alex Caramanico
EDMS 657 HW 4 - Mplus
11/20/2025

Multivariate Mixture Modeling

Take the best fitting model (based on R results) with any constraints placed on the covariance structure and refit this in Mplus using its “constraints” feature. Do your results match that of `mclust()`? Explain.

Mplus code:

```
None
TITLE:Multivariate Mixture Modeling (HW4-1)

DATA:FILE = "Mplus\Iris-Data.dat";

VARIABLE:
  NAMES = sep_len sep_wid pet_len pet_wid species;
  USEVARIABLES ARE sep_len sep_wid pet_len pet_wid;
  CLASSES = c(2);

ANALYSIS:
  TYPE = MIXTURE;
  STARTS = 50 10;
  STSCALE = 5;
  STITERATIONS = 50; ITERATIONS = 1000;
  SDITERATIONS = 250; MITERATIONS = 500;
  MCONVERGENCE = 1E-5;

MODEL:
%overall%
sep_len WITH sep_wid pet_len pet_wid; !comment out to remove covariances
sep_wid WITH pet_len pet_wid; pet_len WITH pet_wid; !comment out to remove
covariances
sep_len*; [sep_len*]; !add matching labels to first term to constrain variance
sep_wid*; [sep_wid*];
pet_len*; [pet_len*];
pet_wid*; [pet_wid*];

%c#1%
sep_len WITH sep_wid pet_len pet_wid; !comment out to remove covariances
sep_wid WITH pet_len pet_wid; pet_len WITH pet_wid; !comment out to remove
covariances
sep_len*; [sep_len*]; !add matching labels to first term to constrain variance
sep_wid*; [sep_wid*];
pet_len*; [pet_len*];
```

```

pet_wid*; [pet_wid*];

%c#2%
sep_len WITH sep_wid pet_len pet_wid; !comment out to remove covariances
sep_wid WITH pet_len pet_wid; pet_len WITH pet_wid; !comment out to remove
covariances
sep_len*; [sep_len*]; !add matching labels to first term to constrain variance
sep_wid*; [sep_wid*];
pet_len*; [pet_len*];
pet_wid*; [pet_wid*];

OUTPUT: TECH1 TECH4; !TECH1: parameters being estimated, TECH8: iter history

```

Note: I spent a significant amount of time trying to implement the “equal shape” constraint in Mplus to fit the 2-class VEV model from R, but found no way of actually constraining that outcome (unless I am missing something obvious). So I fit the next-closest model, which did exactly match the VVV results from R.

	Estimate	Latent Class 2	
Latent Class 1		SEP_LEN WITH	
SEP_LEN WITH		SEP_WID	0.121
SEP_WID	0.097	PET_LEN	0.449
PET_LEN	0.016	PET_WID	0.166
PET_WID	0.010		
		SEP_WID WITH	
SEP_WID WITH		PET_LEN	0.141
PET_LEN	0.011	PET_WID	0.079
PET_WID	0.009		
		PET_LEN WITH	
PET_LEN WITH		PET_WID	0.286
PET_WID	0.006		
Means		Means	
SEP_LEN	5.006	SEP_LEN	6.262
SEP_WID	3.428	SEP_WID	2.872
PET_LEN	1.462	PET_LEN	4.906
PET_WID	0.246	PET_WID	1.676
		Variances	
Variances		SEP_LEN	0.435
SEP_LEN	0.122	SEP_WID	0.110
SEP_WID	0.141	PET_LEN	0.675
PET_LEN	0.030	PET_WID	0.179
PET_WID	0.011		

We see that the results on the left are slightly different to the R results, but only in terms of the estimated covariances and variances. The means are identical. The discrepancies are because the R estimates used VEV for the covariance structure, whereas Mplus used VVV. In other words, Mclust achieved a slightly lower BIC (the model above had a BIC of 574, also matching

Mclust’s VVV model). The overall patterns are still seen in that the covariances between the variables *and* the variances of the variables themselves are much lower (almost) across the board in class 1 than in class 2. This means that the second class has more collinearity between all of its variables, and the variables cover a wider range of values (except for sepal_width)

Latent Profile Analysis (AGOscores.dat dataset)

Four crossed types of goal mastery/performance orientations: (1) mastery-approach, (2) mastery-avoidance, (3) performance-approach, and (4) performance-avoidance for $n=1,868$ students. Using the covariance models A, B, and C (NOT D and E) in Table 1 (p. 17 Pastor (2007), fit 2-, 3-, and 4-class models, allowing means to vary across the classes.

Mplus code template

None

```
TITLE: AGOscores 2-class !rename to 3-class or 4-class accordingly
DATA: file = AGOscores.dat;
VARIABLE:
Names = m_app m_avo p_app p_avo obs;
usevar = m_app m_avo p_app p_avo;
auxiliary = obs;
classes = c(2); !change to c(3) or c(4) accordingly
ANALYSIS:
type = mixture;
starts = 50 25; ! fit each model with 50 random starts, 25 final optimizations
model=NOCOVARIANCES; !remove to include covariances (Model B)
MODEL:
%overall%
!m_app WITH m_avo; m_app WITH p_app; m_app WITH p_avo;
!m_avo WITH p_app; m_avo WITH p_avo;
!p_app WITH p_avo; !add these lines to include covariances (Model B)
[m_app]; [m_avo]; [p_app]; [p_avo]; !mean vector
m_app(s1); m_avo(s1); p_app(s3); p_avo(s4);
!remove blue code to allow disequal variances between each class (Model C)
%c#1%
[m_app]; [m_avo]; [p_app]; [p_avo]; !mean vector
m_app(s1); m_avo(s2); p_app(s3); p_avo(s4);
!remove blue code to allow disequal variances between each class (Model C)
%c#2%
[m_app]; [m_avo]; [p_app]; [p_avo]; !mean vector
m_app(s1); m_avo(s2); p_app(s3); p_avo(s4);
!remove blue code to allow disequal variances between each class (Model C)
%c#3%
[m_app]; [m_avo]; [p_app]; [p_avo]; !mean vector
m_app(s1); m_avo(s2); p_app(s3); p_avo(s4);
!remove blue code to allow disequal variances between each class (Model C)
!add entire chunk to include third class
%c#4%
[m_app]; [m_avo]; [p_app]; [p_avo]; !mean vector
m_app(s1); m_avo(s1); p_app(s1); p_avo(s1);
!remove blue code to allow disequal variances between each class (Model C)
!add entire chunk and the orange one above to include fourth class
```

OUTPUT:

Sampstat Svalues Residual STDYX

Tech1 Tech4 Tech7 Tech11 Tech12 Tech13 Tech14;

PLOT: Type = Plot3; Series = m_app m_avo p_app p_avo (*);

Using SBIC, the Lo, Mendell, & Rubin's adjusted LRT, and entropy (for classification quality), what model fitted the data the best from a statistical perspective? Create your own table to display the results across the different models.

Summary Table

Note: "A" models are constrained to have no covariances between variables, and each variable has equal variance across classes. "B" models allow covariance between each of the variables, but variances are still constrained to be equal across classes. "C" models do not allow any covariance, but variances are class-specific.

<i>Model</i>	<i>SBIC</i>	<i>L-M-R LRT</i>	<i>Entropy</i>
2-class A	40567.339	<0.0001	0.717
2-class B	40115.435	<0.0001	0.822
2-class C	40346.675	<0.0001	0.588
3-class A	40371.865	<0.0001	0.701
3-class B	39903.462	<0.0001	0.775
3-class C	39917.653	0.0156	0.665
4-class A	40100.367	0.0004	0.706
4-class B	39831.065	0.6499	0.709
4-class C	39801.934	<i>Did not converge</i>	0.698

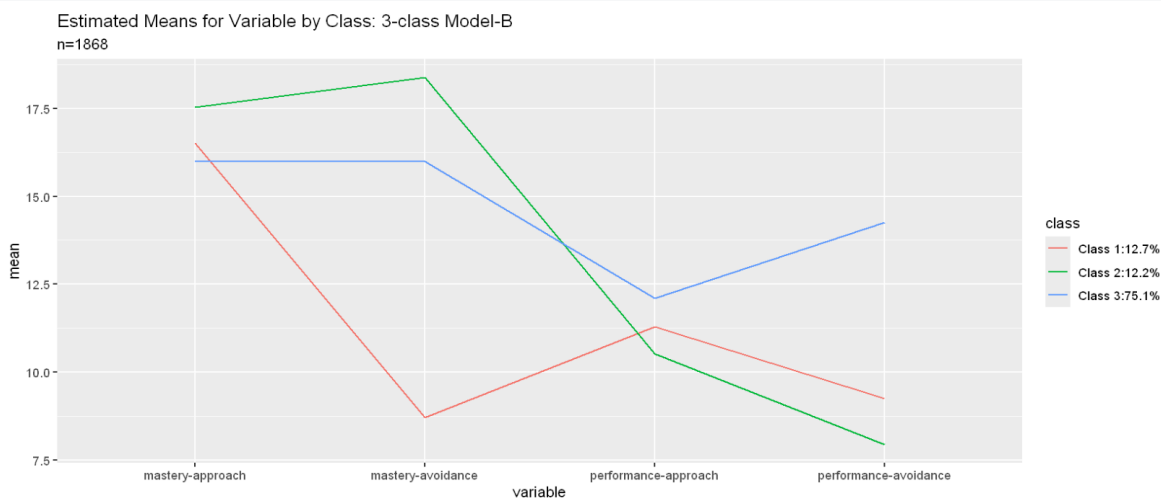
Generally, the "B" models seem to be performing the best as they have among the lowest SBIC and greatest entropy. However, the 4-class "B" model failed to reject the null hypothesis of a significantly improved fit over a 3-class "B" model, so it can be ruled out. This leaves the 2-class and 3-class "B" models, but since the 3-class has a much lower SBIC and rejects the adjusted LRT, it can be selected as the best model (despite it having a lower entropy than the 2-class model). More concretely, the model that seems to be fitting the data the "best" is a 3-class model that constrains variances to be equal across groups, but allows for covariance between the variables. Note that this conclusion is simply based on the models that were actually fit (see "D" and "E" covariance structures, which allow for class-specific variance *and* covariance, and class-specific variance and class-specific covariance, respectively. These models are considerably more complex).

Graph the best fitting model from above. Your graph should include meaningful axes and means of the four variables connected by lines within the same class.

R Code (using Mplus output for values)

```
None
data.frame(class=rep(1:3,each=4) %>%
  factor(labels=c(paste("Class 1:",12.7,"%",sep=""),
    paste("Class 2:",12.2,"%",sep=""),
    paste("Class 3:",75.1,"%",sep=""))),
  variable=rep(c("mastery-approach",
    "mastery-avoidance",
    "performance-approach",
    "performance-avoidance"),
    3),
  mean=c(16.508,8.714,11.288,9.251,
    17.540,18.375,10.511,7.933,
    16.016,15.997,12.103,14.25)) %>%

ggplot() +
  geom_line(mapping=aes(x=variable,y=mean,group=class,color=class)) +
  labs(title="Estimated Means for Variable by Class: 3-class Model-B",
    subtitle="n=1868")
```



From the plot we can ascertain class similarities and differences. The three classes do not vary much in terms of mean mastery-approach or performance-approach, but they vary greatly in terms of mean mastery-avoidance and performance-avoidance. Specifically, class 1 has much lower mastery-avoidance on average, while class 3 has much higher performance-avoidance on average. It is somewhat difficult to see an overall pattern with this type of plot, because the lines intersect each other a fair amount (i.e. no class/group is always on top or always on the bottom). It implies that the data is somewhat challenging to divide into distinct groups, possibly due to a great deal of random noise for certain students. The next prompt is hinted at as well, where the legend shows how frequently students were assigned to class 3

Provide a table for the best fitting model for “Average Latent Class Probabilities for Most Likely Latent Class Membership (Row) by Latent Class (Column)”. Based on the values in the table you create, how well is this model doing in terms of classification? Explain.

Average Latent Class Probabilities for Most Likely Latent Class Membership (Row)
by Latent Class (Column)

	1	2	3
1	0.826	0.013	0.161
2	0.007	0.853	0.140
3	0.037	0.041	0.922

Classification Probabilities for the Most Likely Latent Class Membership (Column)
by Latent Class (Row)

	1	2	3
1	0.785	0.006	0.208
2	0.012	0.762	0.226
3	0.028	0.023	0.949

I included both of the above tables for additional discussion. The first table, “average latent class probabilities...” essentially tells us the certainty with which the model classifies individuals. We generally want large values on the diagonal (near 1) and small values off the diagonal (near 0), which the table seems to satisfy reasonably well. Class 3 appears to be the most “confidently” given label, but this result could also be due to the high number of observations assigned to the group (as seen in other output). This is also why the entries “most likely 2” and “most likely 3” (rows) against 3 as the actual latent class are the largest off-diagonals, since the model “defaults” to an assignment of 3, occasionally making errors. The other off diagonals are very small, suggesting the model is very good at distinguishing between class 1 and 2, but class 3 complicates things.

The second table gives us an idea on measurement error, and essentially communicates the same thing. One of the off-diagonals can be interpreted as “the probability of an observation being assigned to class 3 given it is class 2 is 0.226”. This essentially reinforces the idea just discussed, where the model tends to “overassign” to class 3. This is again supported by the fact the other off-diagonals are all *very* small. Also worth noting is that the probabilities for assigning an individual to latent class 1 or 2 given they belong to the respective class is lower than for latent class 3, because of the favorability toward class 3.